POTATO SURVIVORS — QA 검증 사례

> 직접 만든 Unity 게임을 QA 관점으로 검증한 사례입니다. 엔진 구조와 데이터 흐름을 이해한 상태에서 테스트를 설계하고, 발견한 문제의 원인을 추적해 수정한 뒤 재검증했습니다.

대상	POTATO SURVIVORS (Unity 6 로그라이트 서바이버, Brotato 모작, 2026.08~09)
역할	개발 1인 / QA 설계, 실행, 수정 단독
검증 방식	수동 테스트 케이스 + 자동화 테스트(EditMode) + 디버그 콘솔
도구	Unity Test Framework(NUnit), 스프레드시트, Git/GitHub, Jira(2회차)

QA 프로세스 3단계

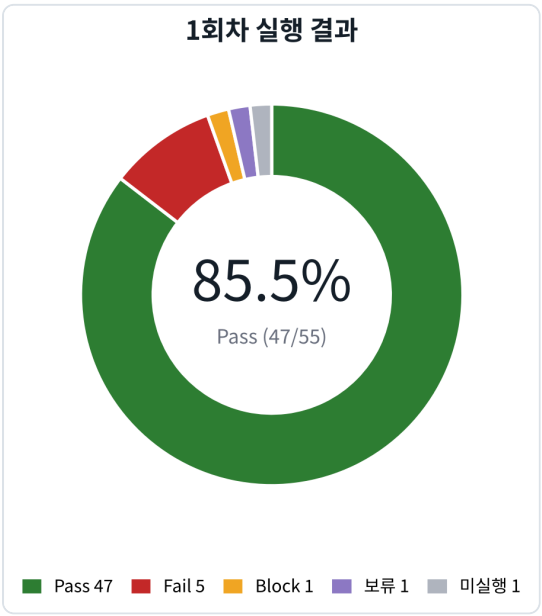
구현과 별개로 품질 검증을 세 층으로 설계했습니다.

1. **수동 테스트 케이스** — 로비, 이동, 전투, 몬스터, 보스, 웨이브, 상점, 세트, HUD 등 전 기능을 아우르는 TC 55개 (사전조건, 절차, 기대결과, 결과, 비고)로 실제 플레이를 검증.
2. **자동화 테스트** — Unity Test Framework로 데미지 공식, 세트 단계, 상점 확률처럼 수치와 확률이 걸린 로직을 플레이 없이 검증. 규칙이 깨지면 바로 실패로 회귀를 감지.
3. **디버그 콘솔(F1)** — 보스전, 만렙, 특정 타이밍처럼 손으로 만들기 어려운 상황을 즉시 재현해 검증 시간을 단축.

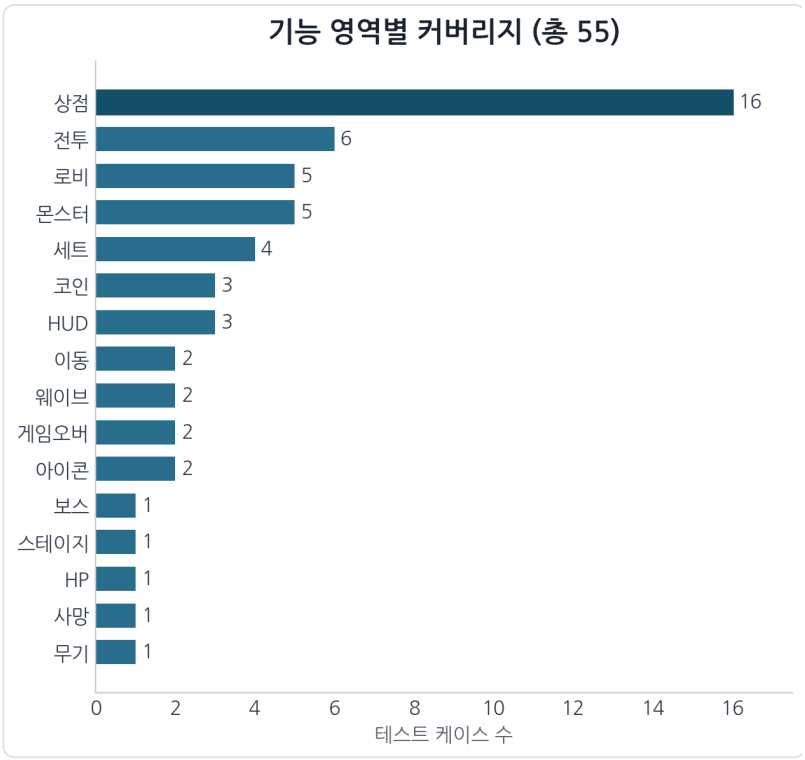
1회차 수치 요약

전체	Pass	Fail	Block	보류	미실행	성공률	진행률
55	47	5	1	1	1	85.5%	98.2%

성공률 = Pass ÷ (전체 − N/A), 진행률 = (전체 − 미실행) ÷ 전체. 1회차는 N/A가 0건이라 성공률은 47/55입니다. Block은 다른 문제 때문에 테스트를 진행할 수 없던 항목, 보류는 사양이 정해지지 않아 판정을 미룬 항목입니다.



1회차 실행 결과



기능 영역별 테스트 케이스 분포 (총 55개)

1회차에서 Pass가 아니었던 항목

전체 55개 TC는 2회차 TC 시트(`potato-qa-tc-round2.xlsx`)의 TC-001~055와 같습니다. 여기서는 1회차에 Pass가 아니었던 항목만 옮겼습니다.

TC	영역	테스트 항목	기대 결과	1회차	처리
TC-007	이동	화면 경계	카메라 화면 경계를 벗어나지 않음	Fail	수정 후 재검증 Pass
TC-018	보스	등장 주기	5스테이지 도달 시 보스 등장	Fail	수정 후 재검증 Pass
TC-028	코인	자석 획득	코인이 흡수되어 골드 증가	Fail	수정 후 재검증 Pass
TC-051	HUD	무기 바	장착 무기 아이콘이 줄로 표시	Fail	수정 후 재검증 Pass
TC-053	아이콘	상점 아이콘	무기 스프라이트가 카드 상단에 표시	Fail	수정 후 재검증 Pass
TC-037	상점	만렙 환불	만렙 무기 구매 시 구매 실패 + 골드 환불	Block	만렙 상태를 수동으로 만들 수 없어 테스트 불가 → 디버그 콘솔로 재현 수단을 만든 뒤 실행, Pass
TC-004	로비	무기 미선택 시작	첫 무기가 기본 선택된 상태로 시작	보류	빈손 시작을 허용할지 사양이 미정 → "첫 무기 기본 선택"으로 확정 후 Pass

미실행 1건은 2회차에서 실행했습니다. 2회차에서는 TC-001~055 전체를 다시 실행했습니다.

발견 버그 상세 (Fail 5건)

TC	문제	원인	수정	재검증
007	화면 끝에서 캐릭터가 절반 잘림	이동 제한이 캐릭터 중심을 화면 경계에 맞춰 클램프	edgePadding만큼 안쪽으로 클램프	Pass
018	5스테이지에 도달해도 보스 미등장	스포너 bossPrefab 인스펙터 연결 누락(빈 참조)	Resources의 SpawnConfig로 런타임 자동 주입, 수동 연결 단계 제거	Pass
028	코인 자석 흡인이 되지 않음	흡인·획득 범위가 지나치게 작음	attractRange, collectRange, moveSpeed 상향	Pass
051	무기 바 3번째 칸 아이콘 미표시	해당 무기 데이터에 아이콘 미연결	무기 에셋에 아이콘 스프라이트 연결	Pass
053	상점 근접무기 아이콘 미표시	진열 목록에 아이콘 없는 구버전 무기 잔존	ShopCatalog 단일 출처화로 구버전 배제	Pass

TC-018과 TC-051은 모두 "에디터에서 손으로 연결하는 단계"를 빠뜨린 같은 원인이었습니다. 018은 연결 단계 자체를 없애 구조적으로 막았고, 051은 아이콘을 연결해 고쳤습니다. 아이콘은 아직 수동 연결이라, 2회차에서 TC-051·054로 같은 누락이 없는지 다시 확인했습니다.

자동화 테스트 (Unity Test Framework)

- 게임 코드를 `Game` 어셈블리로, 테스트를 `GameTests.EditMode` 어셈블리로 분리해 테스트에서 게임 코드를 참조할 수 있는 구조로 전환
- 검증할 로직을 순수 함수(`WeaponMath`, `ShopMath` 등)로 추출해 플레이 없이 빠르게 검증
- 범위: 무기 레벨별 데미지 공식, 관통 증가, 세트 단계(2/4/6), 기본값, 상점 등급 확률 — **EditMode 테스트 36개 모두 통과**
- 효과: 밸런싱이나 리팩터링 중 규칙이 깨지면 즉시 실패로 회귀 감지

디버그 콘솔 (F1)

- 골드 지급, 스테이지 점프, 보스 소환, 무적, 즉사, 배속, 무기 지급을 즉시 조작
- 보스전이나 만렙 환불처럼 손으로 재현하기 오래 걸리는 상황을 몇 초 만에 재현해 검증 시간 단축 (TC-037 Block 해소)
- 에디터/개발 빌드에서만 생성되어 배포 빌드에는 포함되지 않음

확률 검증 자동화 (상점 등급 추첨)

1회차 이후 추가로, 확률이 걸린 상점 등급 추첨을 Unity Test Framework로 자동 검증했습니다. 사람이 플레이로는 확인하기 어려운 확률과 경계 조건을 테스트 코드가 대신 확인하도록 만들었습니다.

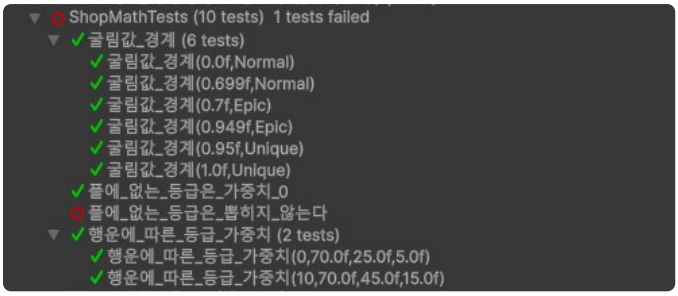
대상	상점 아이템 등급 가중치: Normal 70 / Epic 25 + 행운×2 / Unique 5 + 행운
도구	Unity Test Framework (EditMode), NUnit
테스트 수	ShopMathTests 14개 추가 (전체 36개 모두 통과)
코드	`Assets/Codes/UI/ShopMath.cs`, `Assets/Tests/EditMode/ShopMathTests.cs`

검증 단계

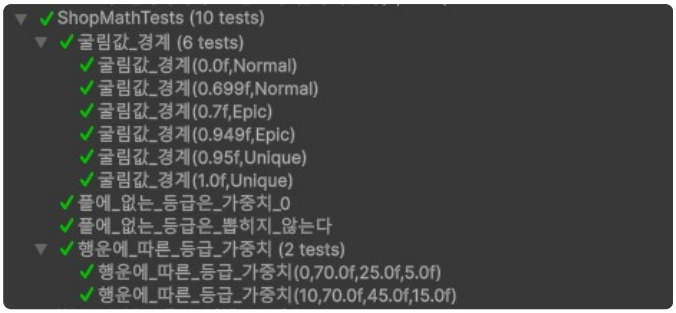
1. **계산 분리** — MonoBehaviour 안에서 `Random.value`를 직접 쓰던 계산을 순수 함수(`ShopMath`)로 추출. 분리 후 기존 테스트 22개가 그대로 통과해 동작 변화가 없음을 확인.
2. **가중치·경계값** — 행운별 가중치 공식, 풀에 없는 등급의 가중치 0, 굴림값 0 / 0.699 / 0.70 / 0.949 / 0.95 / 1.0에서의 등급 전환을 확인.
3. **10만 회 시뮬레이션** — 행운 0, 5, 10, 20에서 각각 10만 회 추첨해 실제 비율을 기획 확률과 비교. 허용 오차는 표준오차의 4배로 계산.

발견 버그

테스트	문제	원인	수정	재검증
풀에 없는 등급은 뽑히지 않는다	Unique가 풀에 없는데 굴림값이 1.0이면 Unique 선택 (기대: Epic)	`Random.value`는 1.0을 포함 → 굴림값 × 합계 = 합계가 되어 어떤 구간에도 들지 않고 마지막 분기(Unique)로 떨어짐	가중치가 있는 등급만 누적하고, 끝까지 가면 가중치가 있는 마지막 등급을 선택	**Pass**



수정 전: 해당 테스트 실패



수정 후: 전체 통과

평균 천만 번에 한 번꼴인 입력이라 플레이로는 재현이 어려운 경우입니다. 상점의 예외 처리 덕분에 멈추지는 않지만, 그 순간 확률 설계와 무관한 아이템이 나오는 문제였습니다.

10만 회 시뮬레이션 결과

12개 항목 모두 허용 오차 안에 들어왔습니다. 행운이 오를수록 Unique 확률이 5%에서 15.6%로 설계대로 올라갑니다.

행운	등급	기획 확률	실제 (10만 회)	차이	허용 오차	판정
0	Normal	70.00%	69.88%	0.12%p	±0.58%p	Pass
0	Epic	25.00%	25.14%	0.14%p	±0.55%p	Pass
0	Unique	5.00%	4.98%	0.02%p	±0.28%p	Pass
5	Normal	60.87%	60.72%	0.15%p	±0.62%p	Pass
5	Epic	30.43%	30.61%	0.18%p	±0.58%p	Pass
5	Unique	8.70%	8.67%	0.03%p	±0.36%p	Pass
10	Normal	53.85%	53.74%	0.11%p	±0.63%p	Pass
10	Epic	34.62%	34.79%	0.17%p	±0.60%p	Pass
10	Unique	11.54%	11.47%	0.07%p	±0.40%p	Pass
20	Normal	43.75%	43.57%	0.18%p	±0.63%p	Pass
20	Epic	40.63%	40.85%	0.22%p	±0.62%p	Pass
20	Unique	15.63%	15.58%	0.05%p	±0.46%p	Pass

이후 진행: 2회차 테스트

1회차 TC 55개에 탐색 TC 10개와 보스 라운드 TC 9개를 더해 2회차(TC 74개)를 진행했습니다. 결과는 Pass 71, Fail 3이며, Fail 3건은 Jira(SCRUM-5·6·7)에 등록해 수정과 재검증까지 추적했습니다. 자세한 내용은 포트폴리오 사이트(ksj000503.github.io)와 2회차 TC 시트에 있습니다.

QA로서 배운 점

구현과 검증을 분리해서 보니, 정상 동작만이 아니라 경계, 연결 누락, 재현하기 어려운 조건에서 문제가 드러난다는 것을 체감했습니다. 보스 미등장과 아이콘 누락은 "수동 연결 의존"이라는 같은 원인에서 나왔고, 보스 쪽은 데이터 주도 구조로 바꿔 재발 가능성을 없앴습니다. 테스트할 수 없던 구조를 순수 함수로 분리하자 플레이로는 천만 번에 한 번 나올 경계 조건을 한 줄의 테스트로 확인할 수 있었고, 확률 검증은 허용 오차라는 판정 기준을 먼저 정해야 한다는 것도 배웠습니다. 수동 TC가 기능 전체를 넓게 보고, 자동 테스트가 수치와 확률을 깊게 보면서 서로의 빈틈을 채운다는 것을 확인했습니다.